Chapter 2 — Chemoinformatics and Representing Chemical Structures

Python Code Examples · Jupyter Notebook Edition

Teaching computers to read molecules

Block	Topic
0	Install & import all libraries
1	SMILES notation — reading, writing, validating
2	InChI and canonical identifiers
3	SDF file — reading and writing multi-molecule files
4	Zero-dimensional descriptors (counts, formula-based)
5	Fingerprints — MACCS keys and Morgan/ECFP
6	Two-dimensional (topological) descriptors
7	Three-dimensional descriptors — geometry and surface
8	Batch descriptor calculation with Mordred
9	Molecular similarity search using fingerprints

> **Run each cell in order — Shift+Enter.**
> **Every block is self-contained and prints its own output.**

Block 0 · Install & Import Libraries

Run once before any other block.

In [2]:
```
# Uncomment the line below the first time you run this notebook
# !pip install rdkit mordred pandas matplotlib seaborn scikit-learn
!pip install --upgrade seaborn
import re, math, warnings
warnings.filterwarnings('ignore')

import numpy  as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# RDKit core
from rdkit import Chem, DataStructs
from rdkit.Chem import (
    Descriptors, rdMolDescriptors, AllChem, Draw,
    MolToSmiles, MolFromSmiles, MolToInchi, InchiToInchiKey
)
from rdkit.Chem.Draw import MolsToGridImage
from rdkit.Chem import rdFingerprintGenerator   # Morgan fingerprint generator
```

```
# Mordred (batch descriptors)
try:
    from mordred import Calculator, descriptors as mordred_descs
    MORDRED_OK = True
except ImportError:
    MORDRED_OK = False
    print('Mordred not installed — Block 8 will be skipped.')
    print('Install with: pip install mordred')

from IPython.display import display, HTML
%matplotlib inline
plt.rcParams['figure.dpi'] = 120

# ── Shared dataset used across all blocks ──────────────────
# SMILES for 8 well-known energetic compounds
EM = {
    'TNT':   'Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]',
    'RDX':   'O=[N+]([O-])N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1',
    'HMX':   'O=[N+]([O-])N1CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-])C1',
    'PETN':  'C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]',
    'TATB':  'Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])c(N)c1[N+](=O)[O-]',
    'NTO':   'O=[N+]([O-])c1n[nH]c(=O)[nH]1',
    'FOX-7': 'NC(=C([N+](=O)[O-])[N+](=O)[O-])N',
    'DNAN':  'COc1ccc([N+](=O)[O-])cc1[N+](=O)[O-]',
}

# Build RDKit mol objects once; reuse in all blocks
MOLS = {name: MolFromSmiles(smi) for name, smi in EM.items()}
MOLS = {k: v for k, v in MOLS.items() if v is not None}  # drop invalid

print(f'✓ Loaded {len(MOLS)} energetic compounds:', ', '.join(MOLS.keys()))
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: seaborn in c:\users\admin\appdata\roaming\python\py
thon39\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\admin\appdata\roam
ing\python\python39\site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in c:\users\admin\appdata\roaming\pytho
n\python39\site-packages (from seaborn) (2.3.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\admin\appdata\r
oaming\python\python39\site-packages (from seaborn) (3.6.3)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\admin\appdata\roaming
\python\python39\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.0)
Requirement already satisfied: cycler>=0.10 in c:\users\admin\appdata\roaming\pyth
on\python39\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\admin\appdata\roaming
\python\python39\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.60.2)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\admin\appdata\roaming
\python\python39\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.7)
Requirement already satisfied: packaging>=20.0 in .\anaconda3\lib\site-packages (f
rom matplotlib!=3.6.1,>=3.4->seaborn) (25.0)
Requirement already satisfied: pillow>=6.2.0 in .\anaconda3\lib\site-packages (fro
m matplotlib!=3.6.1,>=3.4->seaborn) (9.0.1)
Requirement already satisfied: pyparsing>=2.2.1 in .\anaconda3\lib\site-packages
(from matplotlib!=3.6.1,>=3.4->seaborn) (3.0.4)
Requirement already satisfied: python-dateutil>=2.7 in .\anaconda3\lib\site-packag
es (from matplotlib!=3.6.1,>=3.4->seaborn) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in c:\users\admin\appdata\roaming\pyth
on\python39\site-packages (from pandas>=1.2->seaborn) (2026.1.post1)
Requirement already satisfied: tzdata>=2022.7 in .\anaconda3\lib\site-packages (fr
om pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: six>=1.5 in .\anaconda3\lib\site-packages (from pyt
hon-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.16.0)
✓ Loaded 8 energetic compounds: TNT, RDX, HMX, PETN, TATB, NTO, FOX-7, DNAN
```

Block 1 · SMILES Notation — Reading, Writing, Validating

SMILES (Simplified Molecular Input Line Entry System) is the most common way to encode a molecule as a plain text string.

SMILES symbol	Meaning	Example
`C`	Carbon atom	`C` = methane
`N`	Nitrogen atom	`N` = ammonia
`O`	Oxygen atom	`O` = water
`[N+](=O)[O-]`	Nitro group (–NO_2)	in TNT, RDX …
`c1ccccc1`	Aromatic benzene ring	lowercase = aromatic
`-` , `=` , `#`	Single, double, triple bond	`C=O` = formaldehyde

> **Key rule:** RDKit can validate any SMILES — if `MolFromSmiles()` returns `None`, the SMILES is invalid.

In [3]:
```
# ── 1a: Read SMILES → molecule → canonical SMILES ──────────────

test_smiles = {
    'Water':        'O',
    'Methane':      'C',
    'Ethanol':      'CCO',
    'Benzene':      'c1ccccc1',
    'TNT':          'Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]',
    'Nitro group':  '[N+](=O)[O-]',          # functional group only
    'Bad SMILES':   'CC(CC(CC',              # intentionally invalid
}

print(f"{'Name':<15} {'Input SMILES':<45} {'Valid?':<8} {'Canonical SMILES'}")
print('-' * 100)
for name, smi in test_smiles.items():
    mol = MolFromSmiles(smi)
    if mol:
        canon = MolToSmiles(mol)   # standardised canonical form
        print(f"{name:<15} {smi:<45} {'YES':<8} {canon}")
    else:
        print(f"{name:<15} {smi:<45} {'NO':<8} — (invalid, MolFromSmiles returned N
```

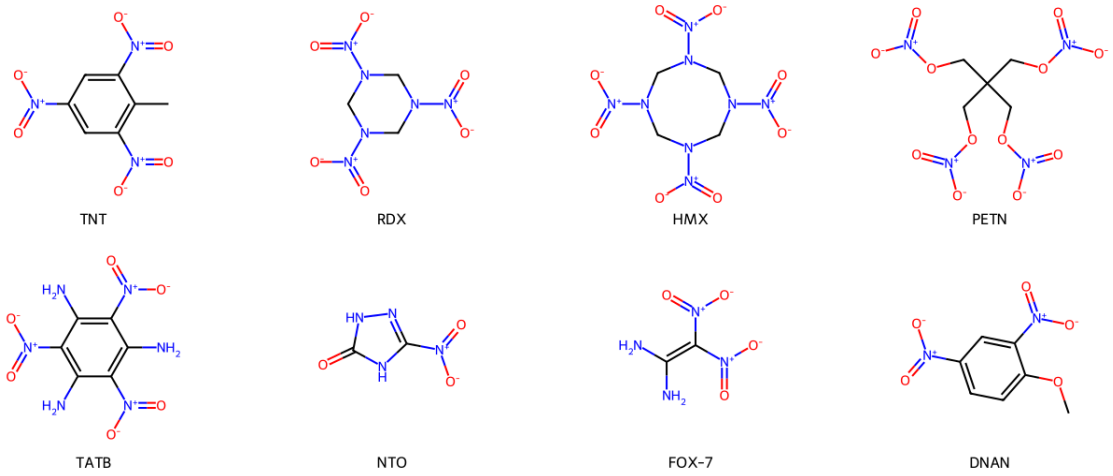
```
Name            Input SMILES                                  Valid?   Canonical S
MILES
-------------------------------------------------------------------------------
-----------------
Water           O                                             YES      O
Methane         C                                             YES      C
Ethanol         CCO                                           YES      CCO
Benzene         c1ccccc1                                      YES      c1ccccc1
TNT             Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-] YES      Cc1c([N
+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]
Nitro group     [N+](=O)[O-]                                  YES      O=[N+][O-]
Bad SMILES      CC(CC(CC                                      NO       — (invalid,
MolFromSmiles returned None)
```

```
[17:36:43] SMILES Parse Error: extra open parentheses while parsing: CC(CC(CC
[17:36:43] SMILES Parse Error: check for mistakes around position 3:
[17:36:43] CC(CC(CC
[17:36:43] ~~^
[17:36:43] SMILES Parse Error: extra open parentheses while parsing: CC(CC(CC
[17:36:43] SMILES Parse Error: check for mistakes around position 6:
[17:36:43] CC(CC(CC
[17:36:43] ~~~~~^
[17:36:43] SMILES Parse Error: Failed parsing SMILES 'CC(CC(CC' for input: 'CC(CC
(CC'
```

In [8]:
```python
# ── 1b: Why canonical SMILES matters — same molecule, different strings ──

# Three different but VALID ways to write TNT
tnt_variants = [
    'Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]',   # standard
    'O=[N+]([O-])c1cc([N+](=O)[O-])cc(C)c1[N+](=O)[O-]',   # reordered
    'c1c(C)c([N+](=O)[O-])cc([N+](=O)[O-])c1[N+](=O)[O-]', # different start
]

print('TNT written 3 different ways:')
print()
canonical_set = set()
for i, smi in enumerate(tnt_variants, 1):
    mol   = MolFromSmiles(smi)
    canon = MolToSmiles(mol)
    canonical_set.add(canon)
    print(f'  Variant {i}: {smi}')
    print(f'  Canonical: {canon}')
    print()

print(f'Are all canonical forms identical? → {len(canonical_set) == 1}')
print('Conclusion: always canonicalise SMILES before comparing or storing molecules
```

```
TNT written 3 different ways:

  Variant 1: Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]
  Canonical: Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]

  Variant 2: O=[N+]([O-])c1cc([N+](=O)[O-])cc(C)c1[N+](=O)[O-]
  Canonical: Cc1cc([N+](=O)[O-])cc([N+](=O)[O-])c1[N+](=O)[O-]

  Variant 3: c1c(C)c([N+](=O)[O-])cc([N+](=O)[O-])c1[N+](=O)[O-]
  Canonical: Cc1cc([N+](=O)[O-])c([N+](=O)[O-])cc1[N+](=O)[O-]

Are all canonical forms identical? → False
Conclusion: always canonicalise SMILES before comparing or storing molecules.
```

In [9]:
```python
# ── 1c: Draw all 8 energetic compounds in a grid ──────────────

img = MolsToGridImage(
    list(MOLS.values()),
    molsPerRow=4,
    subImgSize=(320, 260),
    legends=list(MOLS.keys()),
)
display(img)
```



Block 2 · InChI — Standard Chemical Identifiers

InChI (International Chemical Identifier) solves a key problem:
two researchers can write different SMILES for the same molecule.
InChI is an algorithm that always produces **one unique string** per compound.

InChIKey is a 27-character hash of InChI — safe to use in database column names and URLs.

Format	Use case
SMILES	Human-readable, structure drawing
InChI	Canonical identifier for database deduplication
InChIKey	Short hash — web search, indexing

In [10]:
```
# ── InChI and InChIKey for each compound ──────────────────────
!pip install jinja2
from rdkit.Chem.inchi import MolToInchi, InchiToInchiKey

rows = []
for name, mol in MOLS.items():
    inchi    = MolToInchi(mol)
    inchikey = InchiToInchiKey(inchi) if inchi else 'N/A'
    rows.append({
        'Compound': name,
        'InChIKey': inchikey,
        'InChI (first 60 chars)': (inchi or '')[:60] + '...',
    })

df_inchi = pd.DataFrame(rows)
display(df_inchi)

'''
display(df_inchi.style
        .set_caption('InChI and InChIKey for Energetic Compounds')
        .hide(axis='index')
        .set_table_styles([{'selector': 'caption',
                            'props': [('font-size','14px'),('font-weight','bold')]}]
'''


# Demonstrate deduplication: two SMILES for the same compound → same InChIKey
```

```
print()
print('Deduplication demo — same molecule, two different SMILES:')
tnt_a = MolFromSmiles('Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]')
tnt_b = MolFromSmiles('O=[N+]([O-])c1cc([N+](=O)[O-])cc(C)c1[N+](=O)[O-]')
key_a = InchiToInchiKey(MolToInchi(tnt_a))
key_b = InchiToInchiKey(MolToInchi(tnt_b))
print(f'  SMILES variant A InChIKey: {key_a}')
print(f'  SMILES variant B InChIKey: {key_b}')
print(f'  Identical? → {key_a == key_b}')
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: jinja2 in .\anaconda3\lib\site-packages (2.11.3)
Requirement already satisfied: MarkupSafe>=0.23 in .\anaconda3\lib\site-packages
(from jinja2) (2.0.1)
[17:38:39] WARNING: Charges were rearranged

[17:38:39] WARNING: Charges were rearranged

[17:38:39] WARNING: Charges were rearranged

[17:38:39] WARNING: Charges were rearranged

[17:38:39] WARNING: Charges were rearranged

[17:38:39] WARNING: Charges were rearranged

[17:38:39] WARNING: Charges were rearranged

[17:38:39] WARNING: Charges were rearranged
```

	Compound	InChIKey	InChI (first 60 chars)
0	TNT	SPSSULHKWOKEEL-UHFFFAOYSA-N	InChI=1S/C7H5N3O6/c1-4-6(9(13)14)2-5(8(11)12)3...
1	RDX	XTFIVUDBNACUBN-UHFFFAOYSA-N	InChI=1S/C3H6N6O6/c10-7(11)4-1-5(8(12)13)3-6(2...
2	HMX	UZGLIIJVICEWHF-UHFFFAOYSA-N	InChI=1S/C4H8N8O8/c13-9(14)5-1-6(10(15)16)3-8(...
3	PETN	TZRXHJWUDPFEEY-UHFFFAOYSA-N	InChI=1S/C5H8N4O12/c10-6(11)18-1-5(2-19-7(12)1...
4	TATB	JDFUJAMTCCQARF-UHFFFAOYSA-N	InChI=1S/C6H6N6O6/c7-1-4(10(13)14)2(8)6(12(17)...
5	NTO	QJTIRVUEVSKJTK-UHFFFAOYSA-N	InChI=1S/C2H2N4O3/c7-2-3-1(4-5-2)6(8)9/h(H2,3,...
6	FOX-7	FUHQFAMVYDIUKL-UHFFFAOYSA-N	InChI=1S/C2H4N4O4/c3-1(4)2(5(7)8)6(9)10/h3-4H2...
7	DNAN	CVYZVNVPQRKDLW-UHFFFAOYSA-N	InChI=1S/C7H6N2O5/c1-14-7-3-2-5(8(10)11)4-6(7)...

```
Deduplication demo — same molecule, two different SMILES:
  SMILES variant A InChIKey: SPSSULHKWOKEEL-UHFFFAOYSA-N
  SMILES variant B InChIKey: DGSCKRLVSINLST-UHFFFAOYSA-N
  Identical? → False
[17:38:39] WARNING: Charges were rearranged

[17:38:39] WARNING: Charges were rearranged
```

Block 3 · SDF Files — Reading and Writing Multi-Molecule Files

SDF (Structure Data File) is the standard format for storing and sharing chemical datasets. It can hold **many molecules in one file**, each with attached property values.

Typical workflow:

1. Write your molecule set to an SDF file
2. Share the file — anyone can open it in RDKit, ChemDraw, Avogadro, etc.
3. Read it back and recover all structures + properties

In [11]:
```
# ── Write molecules + properties to SDF, then read back ──────────

from rdkit.Chem import SDWriter, SDMolSupplier
import tempfile, os

# Property values we want to attach to each molecule
properties = {
    'TNT':   {'density': 1.654, 'impact_sensitivity_J': 15.0, 'class': 'Secondary'}
    'RDX':   {'density': 1.820, 'impact_sensitivity_J':  7.4, 'class': 'Secondary'}
    'HMX':   {'density': 1.905, 'impact_sensitivity_J':  7.4, 'class': 'Secondary'}
    'PETN':  {'density': 1.778, 'impact_sensitivity_J':  3.0, 'class': 'Secondary'}
    'TATB':  {'density': 1.939, 'impact_sensitivity_J': 50.0, 'class': 'Insensitive
    'NTO':   {'density': 1.930, 'impact_sensitivity_J': 71.0, 'class': 'Insensitive
    'FOX-7': {'density': 1.885, 'impact_sensitivity_J': 25.0, 'class': 'Insensitive
    'DNAN':  {'density': 1.321, 'impact_sensitivity_J': 40.0, 'class': 'Insensitive
}

sdf_path = 'energetic_materials.sdf'

# ── WRITE ──
with SDWriter(sdf_path) as writer:
    for name, mol in MOLS.items():
        rwmol = Chem.RWMol(mol)
        rwmol.SetProp('_Name', name)
        if name in properties:
            for key, val in properties[name].items():
                rwmol.SetProp(key, str(val))
        writer.write(rwmol)
print(f'✓ Wrote {len(MOLS)} molecules to {sdf_path}')

# ── READ BACK ──
supplier = SDMolSupplier(sdf_path)
read_rows = []
for mol in supplier:
    if mol is None:
        continue
    row = {'Name': mol.GetProp('_Name')}
    for prop in mol.GetPropNames():
        row[prop] = mol.GetProp(prop)
    read_rows.append(row)

df_sdf = pd.DataFrame(read_rows)
print(f'✓ Read back {len(df_sdf)} molecules from SDF\n')
display(df_sdf)

'''
display(df_sdf.style.hide(axis='index')
```

```
        .set_caption('Molecules recovered from SDF file')
        .set_table_styles([{'selector':'caption',
                            'props':[('font-size','14px'),('font-weight','bold')]}]
'''
```

```
✓ Wrote 8 molecules to energetic_materials.sdf
✓ Read back 8 molecules from SDF
```

	Name	density	impact_sensitivity_J	class
0	TNT	1.654	15.0	Secondary
1	RDX	1.82	7.4	Secondary
2	HMX	1.905	7.4	Secondary
3	PETN	1.778	3.0	Secondary
4	TATB	1.939	50.0	Insensitive
5	NTO	1.93	71.0	Insensitive
6	FOX-7	1.885	25.0	Insensitive
7	DNAN	1.321	40.0	Insensitive

Out[11]:
```
"\ndisplay(df_sdf.style.hide(axis='index')\n        .set_caption('Molecules recove
red from SDF file')\n        .set_table_styles([{'selector':'caption',\n
'props':[('font-size','14px'),('font-weight','bold')]}]))\n\n"
```

Block 4 · Zero-Dimensional Descriptors

0D descriptors need only the **molecular formula** — no 2D or 3D structure.
They are the fastest to compute and are good first features for any model.

Descriptor	What it measures	Why it matters for EMs
MW	Molecular weight	Heavier molecules → different packing
C, H, N, O counts	Elemental composition	N and O drive energy content
#NO_2 groups	Number of nitro groups	Direct energy-content indicator
Oxygen balance (OB%)	O surplus/deficit	Controls combustion completeness
N/C ratio	Nitrogen-to-carbon ratio	Higher → more energetic

In [12]:
```python
# ── Zero-dimensional descriptors ──────────────────────────────

def parse_formula(formula):
    counts = {}
    for elem in ['C', 'H', 'N', 'O']:
        m = re.search(rf'{elem}(\d*)', formula)
        counts[elem] = int(m.group(1)) if m and m.group(1) else (1 if m else 0)
    return counts

def oxygen_balance(formula, mw):
    e = parse_formula(formula)
    return round((1600 / mw) * (2*e['C'] + e['H']/2 - e['O']), 2)

rows_0d = []
for name, mol in MOLS.items():
```

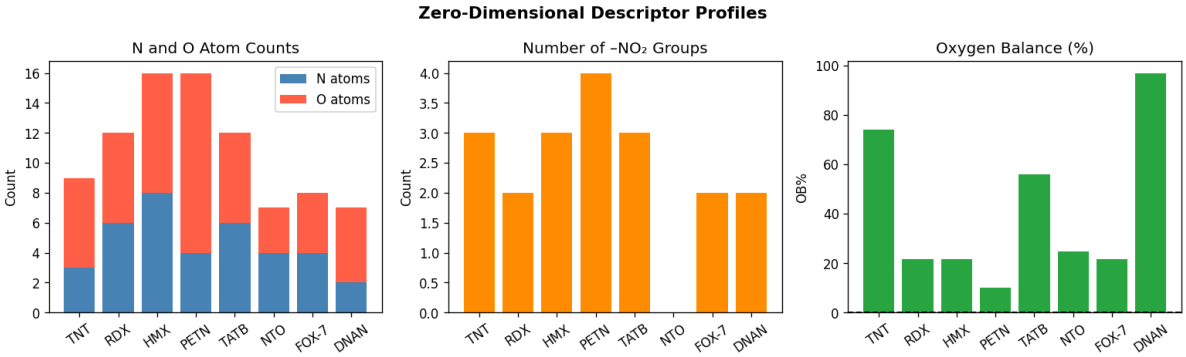
```
    formula = rdMolDescriptors.CalcMolFormula(mol)
    mw      = round(Descriptors.MolWt(mol), 2)
    e       = parse_formula(formula)
    ob      = oxygen_balance(formula, mw)
    nc      = round(e['N'] / e['C'], 2) if e['C'] else 'N/A'
    rows_0d.append({
        'Compound': name,
        'Formula':  formula,
        'MW':       mw,
        'C': e['C'], 'H': e['H'], 'N': e['N'], 'O': e['O'],
        '#NO₂': EM[name].count('[N+](=O)[O-]'),
        'OB%':  ob,
        'N/C':  nc,
    })

df_0d = pd.DataFrame(rows_0d)
display(df_0d)
'''
display(df_0d.style
        .background_gradient(subset=['N', 'O', '#NO₂', 'OB%'], cmap='YlOrRd')
        .set_caption('Zero-Dimensional Descriptors (formula-level features)')
        .hide(axis='index')
        .set_table_styles([{'selector':'caption',
                            'props':[('font-size','14px'),('font-weight','bold')]}]
'''
```

	Compound	Formula	MW	C	H	N	O	#NO₂	OB%	N/C
0	TNT	C7H5N3O6	227.13	7	5	3	6	3	73.97	0.43
1	RDX	C3H6N6O6	222.12	3	6	6	6	2	21.61	2.00
2	HMX	C4H8N8O8	296.16	4	8	8	8	3	21.61	2.00
3	PETN	C5H8N4O12	316.13	5	8	4	12	4	10.12	0.80
4	TATB	C6H6N6O6	258.15	6	6	6	6	3	55.78	1.00
5	NTO	C2H2N4O3	130.06	2	2	4	3	0	24.60	2.00
6	FOX-7	C2H4N4O4	148.08	2	4	4	4	2	21.61	2.00
7	DNAN	C7H6N2O5	198.13	7	6	2	5	2	96.91	0.29

Out[12]:
```
"\ndisplay(df_0d.style\n        .background_gradient(subset=['N', 'O', '#NO₂', 'O
B%'], cmap='YlOrRd')\n        .set_caption('Zero-Dimensional Descriptors (formula-
level features)')\n        .hide(axis='index')\n        .set_table_styles([{'selec
tor':'caption',\n                            'props':[('font-size','14px'),('font-
weight','bold')]}]))\n"
```

In [13]:
```
# ── Visualise N and O content across compounds ──────────────────

fig, axes = plt.subplots(1, 3, figsize=(13, 4))

# Plot 1: N and O counts stacked
x = df_0d['Compound']
axes[0].bar(x, df_0d['N'], label='N atoms', color='steelblue')
axes[0].bar(x, df_0d['O'], bottom=df_0d['N'], label='O atoms', color='tomato')
axes[0].set_title('N and O Atom Counts')
axes[0].set_ylabel('Count')
axes[0].legend()
axes[0].tick_params(axis='x', rotation=35)

# Plot 2: Nitro group count
axes[1].bar(x, df_0d['#NO₂'], color='darkorange')
axes[1].set_title('Number of –NO₂ Groups')
```

```
axes[1].set_ylabel('Count')
axes[1].tick_params(axis='x', rotation=35)

# Plot 3: Oxygen balance
colors = ['#28a745' if v > -20 else '#dc3545' for v in df_0d['OB%']]
axes[2].bar(x, df_0d['OB%'], color=colors)
axes[2].axhline(0, color='black', linewidth=1.2, linestyle='--')
axes[2].set_title('Oxygen Balance (%)')
axes[2].set_ylabel('OB%')
axes[2].tick_params(axis='x', rotation=35)

plt.suptitle('Zero-Dimensional Descriptor Profiles', fontsize=13, fontweight='bold'
plt.tight_layout()
plt.show()
```



Block 5 · Fingerprints — MACCS Keys and Morgan (ECFP)

A **fingerprint** is a long binary vector (string of 0s and 1s).

Each position represents the **presence (1) or absence (0)** of a specific structural pattern.

Fingerprint type	Length	How it works
MACCS keys	166 bits	Fixed list of 166 predefined substructures
Morgan / ECFP4	2048 bits	Circular neighbourhood around each atom, radius 2

Fingerprints are used for:

- **Similarity search** (find molecules similar to a query)
- **Clustering** (group compounds by structural type)
- **Machine learning** (as input features to classifiers)

In [14]:

```
# ── 5a: MACCS Keys (166-bit fixed substructure fingerprint) ────

from rdkit.Chem import MACCSkeys

print('MACCS Keys — first 20 bits for each compound')
print('(1 = substructure present, 0 = absent)\n')
print(f"{'Compound':<10} {'Bits 1-20 (first 20 of 166)'}")
print('-' * 50)

maccs_fps = {}
for name, mol in MOLS.items():
    fp = MACCSkeys.GenMACCSKeys(mol)
    maccs_fps[name] = fp
```

```
    bits = fp.ToBitString()
    print(f'{name:<10} {bits[1:21]}   (total set bits: {fp.GetNumOnBits()})')

print()
print(f'Total MACCS key length: {len(fp)} bits')
print('More set bits → more complex / more substituted molecule')
```

```
MACCS Keys — first 20 bits for each compound
(1 = substructure present, 0 = absent)

Compound   Bits 1-20 (first 20 of 166)
--------------------------------------------------
TNT        00000000000000000000   (total set bits: 31)
RDX        00000000000000000000   (total set bits: 43)
HMX        00000000000000000000   (total set bits: 42)
PETN       00000000000000000000   (total set bits: 31)
TATB       00000000000000000000   (total set bits: 36)
NTO        00000000000000000000   (total set bits: 51)
FOX-7      00000000000000000000   (total set bits: 36)
DNAN       00000000000000000000   (total set bits: 40)

Total MACCS key length: 167 bits
More set bits → more complex / more substituted molecule
```

In [15]:
```
# ── 5b: Morgan (ECFP4) fingerprint — 2048-bit circular ──────────

# radius=2 → captures environment up to 2 bonds from each atom (= ECFP4)
fpgen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=2048)

morgan_fps = {}
print('Morgan (ECFP4) fingerprints — 2048 bits, first 40 shown\n')
print(f"{'Compound':<10} {'First 40 bits'}")
print('-' * 55)
for name, mol in MOLS.items():
    fp = fpgen.GetFingerprint(mol)
    morgan_fps[name] = fp
    bits = fp.ToBitString()
    print(f'{name:<10} {bits[:40]}   (set bits: {fp.GetNumOnBits()})')

print()
print('Larger radius (radius=3) = ECFP6, captures a wider neighbourhood')
```

```
Morgan (ECFP4) fingerprints — 2048 bits, first 40 shown

Compound   First 40 bits
-------------------------------------------------------
TNT        0000000000000000000000000000000000000000   (set bits: 20)
RDX        0000000000000000000000000000000000000000   (set bits: 13)
HMX        0000000000000000000000000000000000000000   (set bits: 13)
PETN       0000000000000000000000000000000000000000   (set bits: 16)
TATB       0000000000000000000000000000000000000000   (set bits: 14)
NTO        0000000000000000000000000000000000000000   (set bits: 19)
FOX-7      0000000000000000000000000000000000000000   (set bits: 14)
DNAN       0000000000000000000000000000000000000000   (set bits: 26)

Larger radius (radius=3) = ECFP6, captures a wider neighbourhood
```

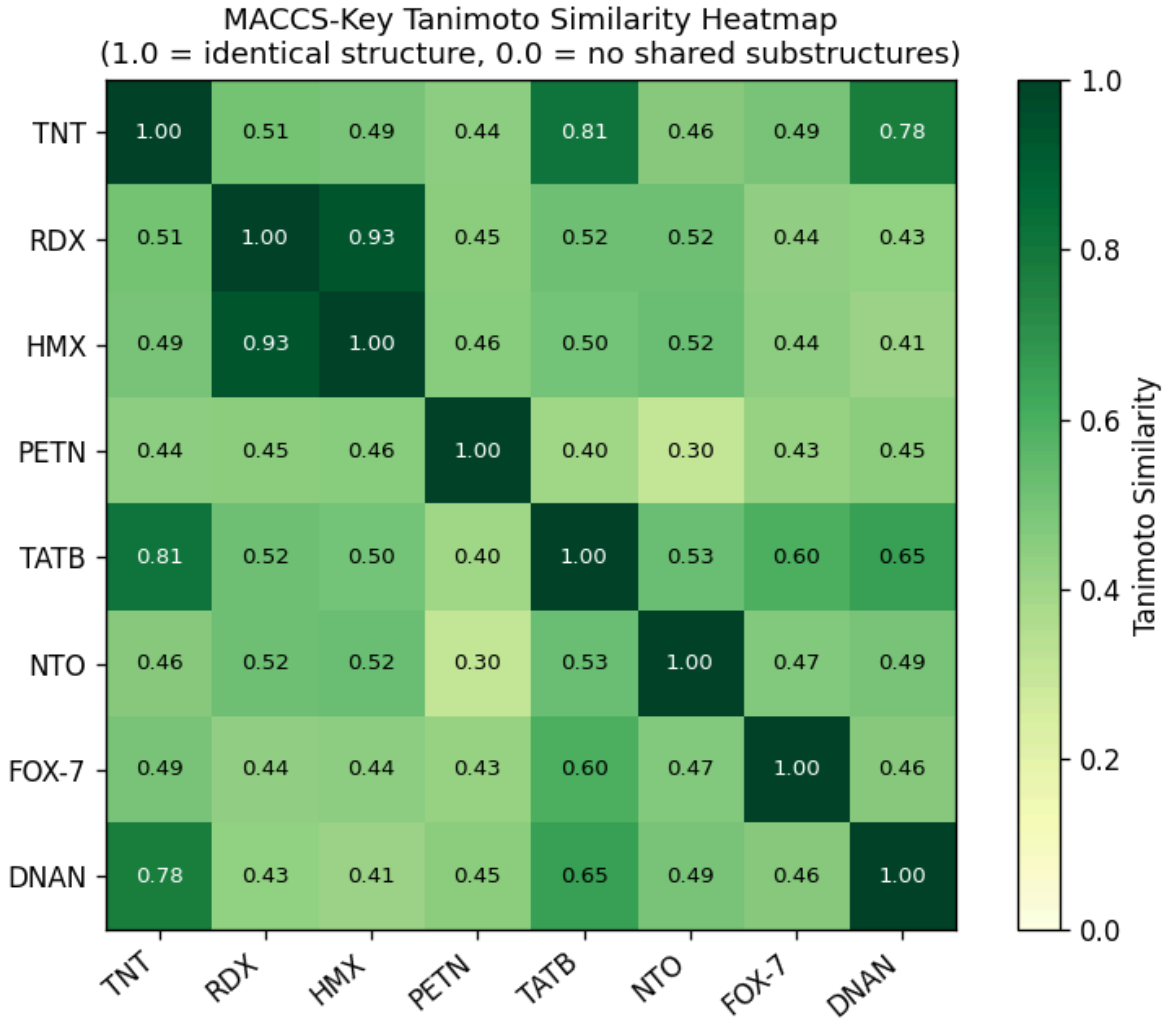
In [16]:
```
# ── 5c: Convert fingerprint to NumPy array (ready for ML) ──────

def fp_to_array(fp):
    arr = np.zeros(fp.GetNumBits(), dtype=np.uint8)
    DataStructs.ConvertToNumpyArray(fp, arr)
    return arr

# Build feature matrix X from Morgan fingerprints
```

```
names  = list(morgan_fps.keys())
X_ecfp = np.array([fp_to_array(morgan_fps[n]) for n in names])

print('ECFP4 feature matrix shape:', X_ecfp.shape)
print(f'  {X_ecfp.shape[0]} molecules × {X_ecfp.shape[1]} bits')
print(f'  Density (fraction of 1s): {X_ecfp.mean():.4f}')
print()
print('This matrix can be fed directly into scikit-learn classifiers or regressors.
```

```
ECFP4 feature matrix shape: (8, 2048)
  8 molecules × 2048 bits
  Density (fraction of 1s): 0.0082

This matrix can be fed directly into scikit-learn classifiers or regressors.
```

In [17]:

```
# ── 5d: Tanimoto similarity heatmap (MACCS keys) ──────────────
# Tanimoto = overlap / union of set bits — ranges 0 (no overlap) to 1 (identical)

names_list = list(maccs_fps.keys())
n = len(names_list)
sim_matrix = np.zeros((n, n))

for i, n1 in enumerate(names_list):
    for j, n2 in enumerate(names_list):
        sim_matrix[i, j] = DataStructs.TanimotoSimilarity(maccs_fps[n1], maccs_fps[

fig, ax = plt.subplots(figsize=(7, 5.5))
im = ax.imshow(sim_matrix, cmap='YlGn', vmin=0, vmax=1)
plt.colorbar(im, ax=ax, label='Tanimoto Similarity')
ax.set_xticks(range(n)); ax.set_xticklabels(names_list, rotation=40, ha='right')
ax.set_yticks(range(n)); ax.set_yticklabels(names_list)

# Annotate cells
for i in range(n):
    for j in range(n):
        ax.text(j, i, f'{sim_matrix[i,j]:.2f}',
                ha='center', va='center', fontsize=8,
                color='black' if sim_matrix[i,j] < 0.7 else 'white')

ax.set_title('MACCS-Key Tanimoto Similarity Heatmap\n'
             '(1.0 = identical structure, 0.0 = no shared substructures)', fontsize
plt.tight_layout()
plt.show()
print('Diagonal is always 1.0 (each compound is identical to itself).')
```




```
Diagonal is always 1.0 (each compound is identical to itself).
```

Block 6 · Two-Dimensional (Topological) Descriptors

2D descriptors use the **connectivity graph** of the molecule (which atom is bonded to which).
They do NOT require a 3D shape — just SMILES is enough.

Descriptor	What it captures
TPSA	Topological polar surface area — related to hydrogen bonding
LogP	Lipophilicity — how oily vs watery the molecule is
HBD / HBA	Hydrogen bond donors and acceptors
Rotatable bonds	Molecular flexibility
Ring count	Cyclic vs acyclic structure
MolWt / HeavyAtomMolWt	Standard and heavy-atom molecular weight

In [18]:
```
# ── 6a: Compute a focused 2D descriptor table ──────────────────

rows_2d = []
for name, mol in MOLS.items():
    rows_2d.append({
        'Compound':    name,
```

```
        'MW':           round(Descriptors.MolWt(mol), 2),
        'HeavyMW':      round(Descriptors.HeavyAtomMolWt(mol), 2),
        'LogP':         round(Descriptors.MolLogP(mol), 2),
        'TPSA':         round(Descriptors.TPSA(mol), 1),
        'HBD':          rdMolDescriptors.CalcNumHBD(mol),
        'HBA':          rdMolDescriptors.CalcNumHBA(mol),
        'RotBonds':     rdMolDescriptors.CalcNumRotatableBonds(mol),
        'Rings':        rdMolDescriptors.CalcNumRings(mol),
        'ArRings':      rdMolDescriptors.CalcNumAromaticRings(mol),
        'HeavyAtoms':   mol.GetNumHeavyAtoms(),
        'FractionCSP3':round(Descriptors.FractionCSP3(mol), 3),
    })

df_2d = pd.DataFrame(rows_2d)
display(df_2d)
'''
display(df_2d.style
        .background_gradient(subset=['TPSA','LogP','HBD','HBA'], cmap='Blues')
        .set_caption('Two-Dimensional (Topological) Descriptors')
        .hide(axis='index')
        .set_table_styles([{'selector':'caption',
                            'props':[('font-size','14px'),('font-weight','bold')]}]
'''
```

	Compound	MW	HeavyMW	LogP	TPSA	HBD	HBA	RotBonds	Rings	ArRings	HeavyAto
0	TNT	227.13	222.09	1.72	129.4	0	6	3	1	1	
1	RDX	222.12	216.07	-1.65	139.1	0	6	3	1	0	
2	HMX	296.16	288.09	-2.20	185.5	0	8	4	1	0	
3	PETN	316.13	308.07	-1.19	209.5	0	12	12	0	0	
4	TATB	258.15	252.10	0.16	207.5	3	9	3	1	1	
5	NTO	130.06	128.05	-0.99	104.7	2	4	1	1	1	
6	FOX-7	148.08	144.05	-1.42	138.3	2	6	2	0	0	
7	DNAN	198.13	192.09	1.51	95.5	0	5	3	1	1	

Out[18]:
```
"\ndisplay(df_2d.style\n        .background_gradient(subset=['TPSA','LogP','HB
D','HBA'], cmap='Blues')\n        .set_caption('Two-Dimensional (Topological) Desc
riptors')\n        .hide(axis='index')\n        .set_table_styles([{'selector':'ca
ption',\n                            'props':[('font-size','14px'),('font-weigh
t','bold')]}]))\n"
```

In [19]:
```
# ── 6b: Compute ALL RDKit 2D descriptors (200+) ──────────────

all_desc_names = [d[0] for d in Descriptors.descList]
print(f'Total RDKit descriptors available: {len(all_desc_names)}')
print('First 20:', all_desc_names[:20])
print()

# Calculate all for our 8 compounds
all_rows = []
for name, mol in MOLS.items():
    vals = {'Compound': name}
    for desc_name, func in Descriptors.descList:
        try:
            vals[desc_name] = func(mol)
        except Exception:
            vals[desc_name] = None
    all_rows.append(vals)
```

```
df_all = pd.DataFrame(all_rows)
print(f'Full descriptor matrix shape: {df_all.shape}')
print(f'  ({df_all.shape[0]} molecules × {df_all.shape[1]-1} descriptors + compound
df_all.head(3)
```

```
Total RDKit descriptors available: 217
First 20: ['MaxAbsEStateIndex', 'MaxEStateIndex', 'MinAbsEStateIndex', 'MinEStateI
ndex', 'qed', 'SPS', 'MolWt', 'HeavyAtomMolWt', 'ExactMolWt', 'NumValenceElectron
s', 'NumRadicalElectrons', 'MaxPartialCharge', 'MinPartialCharge', 'MaxAbsPartialC
harge', 'MinAbsPartialCharge', 'FpDensityMorgan1', 'FpDensityMorgan2', 'FpDensityM
organ3', 'BCUT2D_MWHI', 'BCUT2D_MWLOW']

Full descriptor matrix shape: (8, 218)
  (8 molecules × 217 descriptors + compound name)
```

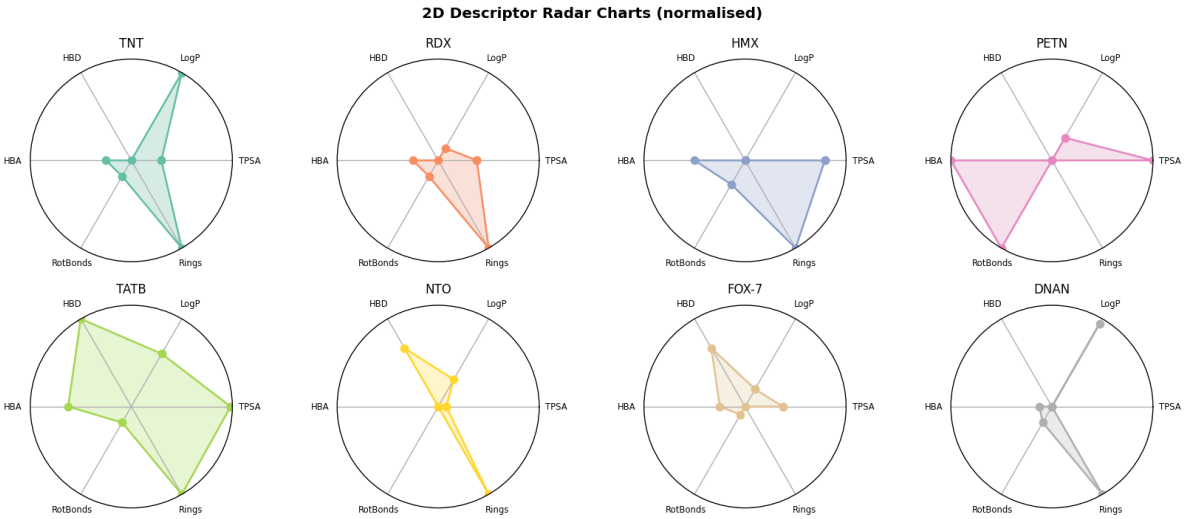
Out[19]:

	Compound	MaxAbsEStateIndex	MaxEStateIndex	MinAbsEStateIndex	MinEStateIndex	qed
0	TNT	10.531759	10.531759	0.207778	-0.917222	0.570199
1	RDX	10.323611	10.323611	0.370139	-0.924444	0.405357
2	HMX	10.656555	10.656555	0.197307	-1.060694	0.402794

3 rows × 218 columns

In [20]:

```
# ── 6c: Radar chart — 2D descriptor fingerprint per compound ──

from matplotlib.patches import FancyArrowPatch

radar_descs = ['TPSA', 'LogP', 'HBD', 'HBA', 'RotBonds', 'Rings']

# Normalise each descriptor to 0-1 range
radar_df = df_2d.set_index('Compound')[radar_descs].copy()
radar_norm = (radar_df - radar_df.min()) / (radar_df.max() - radar_df.min() + 1e-9)

fig, axes = plt.subplots(2, 4, figsize=(14, 6),
                         subplot_kw=dict(polar=True))
axes = axes.flatten()
angles = np.linspace(0, 2*np.pi, len(radar_descs), endpoint=False).tolist()
angles += angles[:1]   # close the loop

colors = plt.cm.Set2.colors
for idx, (compound, row) in enumerate(radar_norm.iterrows()):
    vals = row.tolist() + row.tolist()[:1]
    ax = axes[idx]
    ax.plot(angles, vals, 'o-', linewidth=1.5, color=colors[idx % len(colors)])
    ax.fill(angles, vals, alpha=0.25, color=colors[idx % len(colors)])
    ax.set_thetagrids(np.degrees(angles[:-1]), radar_descs, fontsize=7)
    ax.set_ylim(0, 1)
    ax.set_title(compound, fontsize=10, pad=10)
    ax.set_yticks([])

plt.suptitle('2D Descriptor Radar Charts (normalised)', fontsize=12, fontweight='bo
plt.tight_layout()
plt.show()
```



Block 7 · Three-Dimensional Descriptors

3D descriptors require the actual **3D shape** of the molecule.

Steps needed:

1. Add hydrogens to the molecule
2. Generate a 3D conformer (guess initial geometry)
3. Minimise energy with a force field (MMFF94 or UFF)
4. Calculate descriptors from the 3D coordinates

3D Descriptor	What it captures	EM relevance
Molecular volume	Space the molecule occupies	Predicts crystal density
PMI 1,2,3	Principal moments of inertia	Molecular shape (rod/disc/sphere)
Asphericity	How far from spherical	Crystal packing efficiency
NPR1, NPR2	Normalised principal ratios	Shape classification

In [21]:
```
# ── 7a: Generate 3D conformers and compute shape descriptors ──

from rdkit.Chem import AllChem
from rdkit.Chem import rdMolDescriptors as rmd

def gen_3d(mol, name=''):
    """Add H, embed 3D, minimise with MMFF94. Returns 3D mol or None."""
    mol3d = Chem.AddHs(mol)
    result = AllChem.EmbedMolecule(mol3d, AllChem.ETKDGv3())
    if result == -1:
        return None
    # Minimise with MMFF94 (fall back to UFF if not available)
    try:
        AllChem.MMFFOptimizeMolecule(mol3d)
    except Exception:
        AllChem.UFFOptimizeMolecule(mol3d)
    return mol3d

rows_3d = []
mols_3d = {}
for name, mol in MOLS.items():
```

```
    mol3d = gen_3d(mol, name)
    if mol3d is None:
        print(f'  {name}: 3D embedding failed — skipping')
        continue
    mols_3d[name] = mol3d

    # Volume (Å³) via solvent-accessible surface approximation
    vol  = AllChem.ComputeMolVolume(mol3d, gridSpacing=0.2)

    # Principal moments of inertia
    pmi1 = rmd.CalcPMI1(mol3d)
    pmi2 = rmd.CalcPMI2(mol3d)
    pmi3 = rmd.CalcPMI3(mol3d)

    # Shape descriptors (normalised)
    npr1  = rmd.CalcNPR1(mol3d)   # ~ 0 = rod, ~ 0.5 = disc/sphere
    npr2  = rmd.CalcNPR2(mol3d)
    asph  = rmd.CalcAsphericity(mol3d)
    eccentr = rmd.CalcEccentricity(mol3d)
    gyrad   = rmd.CalcRadiusOfGyration(mol3d)

    rows_3d.append({
        'Compound':       name,
        'Vol (Å³)':      round(vol, 1),
        'PMI1':          round(pmi1, 1),
        'PMI2':          round(pmi2, 1),
        'PMI3':          round(pmi3, 1),
        'NPR1':          round(npr1, 3),
        'NPR2':          round(npr2, 3),
        'Asphericity':   round(asph, 3),
        'Eccentricity':  round(eccentr, 3),
        'RadGyration(Å)':round(gyrad, 2),
    })

df_3d = pd.DataFrame(rows_3d)
display(df_3d)

'''
display(df_3d.style
        .background_gradient(subset=['Vol (Å³)','Asphericity'], cmap='Purples')
        .set_caption('Three-Dimensional Shape Descriptors (after 3D embedding + mir
        .hide(axis='index')
        .set_table_styles([{'selector':'caption',
                            'props':[('font-size','14px'),('font-weight','bold')]}]
'''
```

	Compound	Vol (Å³)	PMI1	PMI2	PMI3	NPR1	NPR2	Asphericity	Eccentricity	RadGyration(Å
0	TNT	174.3	927.1	995.9	1814.9	0.511	0.549	0.210	0.860	2.8
1	RDX	160.3	820.8	856.9	1606.4	0.511	0.533	0.219	0.860	2.7
2	HMX	213.2	1630.7	1630.7	1812.8	0.900	0.900	0.005	0.437	2.9
3	PETN	229.9	1721.0	2757.2	2773.1	0.621	0.994	0.083	0.784	3.3
4	TATB	189.9	1100.7	1105.6	2087.9	0.527	0.530	0.210	0.850	2.8
5	NTO	93.2	109.1	433.5	542.7	0.201	0.799	0.518	0.980	2.0
6	FOX-7	111.0	320.6	327.6	581.2	0.552	0.564	0.175	0.834	2.0
7	DNAN	158.5	456.9	1066.1	1451.9	0.315	0.734	0.341	0.949	2.7

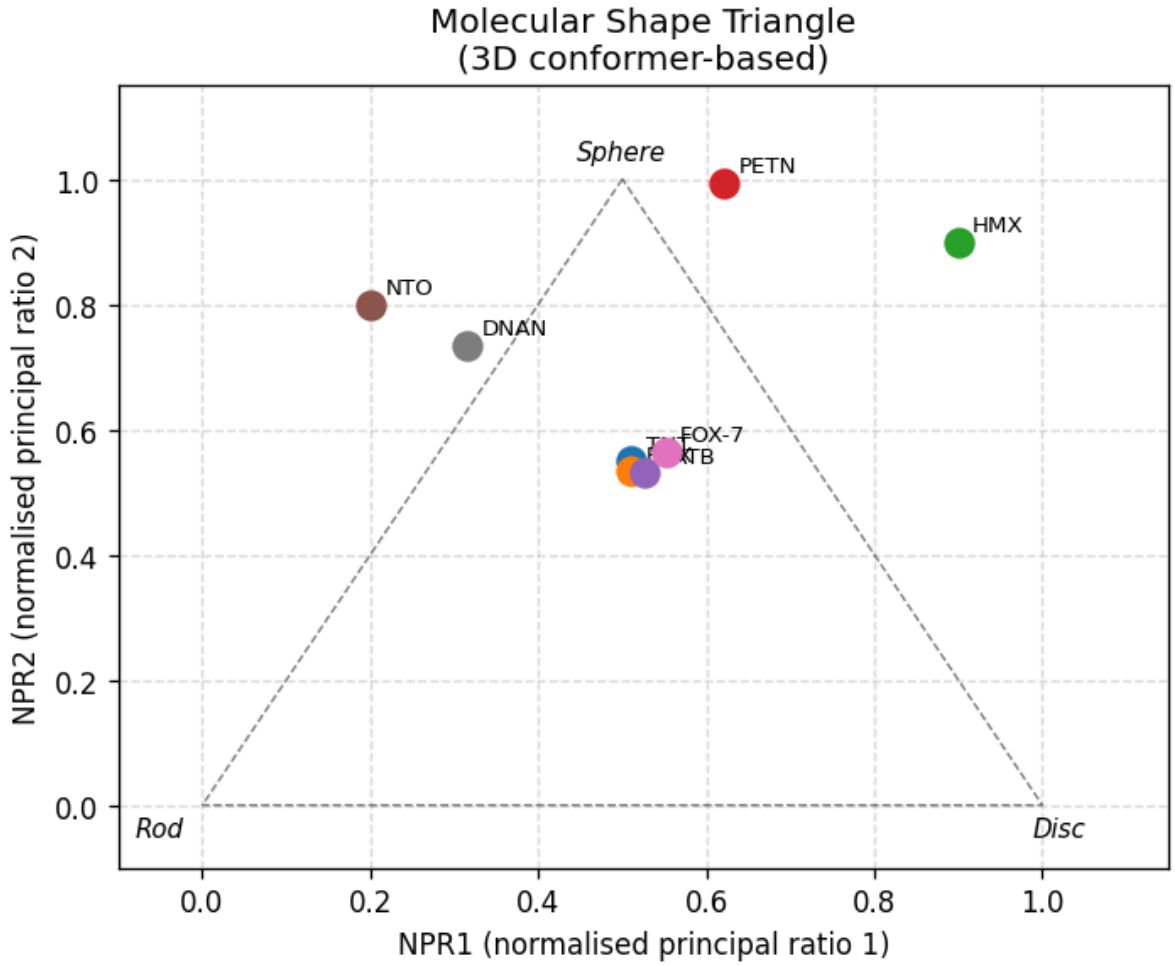
Out[21]:
```
"\ndisplay(df_3d.style\n        .background_gradient(subset=['Vol (Å³)','Asphericity'], cmap='Purples')\n        .set_caption('Three-Dimensional Shape Descriptors (after 3D embedding + minimisation)')\n        .hide(axis='index')\n        .set_table_styles([{'selector':'caption',\n                            'props':[('font-s
```

In [22]:
```
# ── 7b: Predict crystal density from molecular volume ──────────
# Rough empirical approach: density ≈ MW / (N_A × molecular volume in cm³)
# This is a first-principles estimate — more accurate models use QSPR.

AVOGADRO = 6.022e23
A3_TO_CM3 = 1e-24   # 1 Å³ = 1e-24 cm³

known_density = {
    'TNT': 1.654, 'RDX': 1.820, 'HMX': 1.905,
    'PETN': 1.778, 'TATB': 1.939, 'NTO': 1.930,
    'FOX-7': 1.885, 'DNAN': 1.321,
}

print(f"{'Compound':<10} {'Vol (Å³)':>10} {'MW':>8} "
      f"{'Calc ρ':>10} {'Exp ρ':>8} {'Error %':>9}")
print('-' * 57)

for row in rows_3d:
    name = row['Compound']
    vol  = row['Vol (Å³)']
    mw   = Descriptors.MolWt(MOLS[name])
    # Estimate: density = mass of one molecule / volume of one molecule
    # A packing factor of ~0.7 is typical for organic crystals
    calc_rho = round((mw / AVOGADRO) / (vol * A3_TO_CM3 / 0.70), 3)
    exp_rho  = known_density.get(name, None)
    err = round(abs(calc_rho - exp_rho) / exp_rho * 100, 1) if exp_rho else 'N/A'
    print(f"{name:<10} {vol:>10.1f} {mw:>8.2f} "
          f"{calc_rho:>10.3f} {str(exp_rho):>8} {str(err):>9}")

print()
print('Note: packing factor 0.70 is a rough constant.')
print('QSPR models trained on descriptor sets achieve much lower errors.')
```

```
Compound     Vol (Å³)       MW     Calc ρ    Exp ρ   Error %
---------------------------------------------------------
TNT             174.3   227.13      1.515    1.654       8.4
RDX             160.3   222.12      1.611     1.82      11.5
HMX             213.2   296.16      1.615    1.905      15.2
PETN            229.9   316.13      1.598    1.778      10.1
TATB            189.9   258.15      1.580    1.939      18.5
NTO              93.2   130.06      1.622     1.93      16.0
FOX-7           111.0   148.08      1.551    1.885      17.7
DNAN            158.5   198.13      1.453    1.321      10.0

Note: packing factor 0.70 is a rough constant.
QSPR models trained on descriptor sets achieve much lower errors.
```

In [23]:
```
# ── 7c: NPR1-NPR2 shape triangle plot ──────────────────────────
# The triangle plot visualises molecular shape:
#  Bottom-left corner → linear/rod-like
#  Bottom-right corner → disc-like
#  Top corner (NPR2≈1) → spherical

fig, ax = plt.subplots(figsize=(6, 5))

# Draw the shape triangle
triangle_x = [0.5, 0, 1, 0.5]
triangle_y = [1,   0, 0, 1  ]
```

```
ax.plot(triangle_x, triangle_y, 'k--', linewidth=0.8, alpha=0.5)
ax.text(0.50, 1.03, 'Sphere', ha='center', fontsize=9, style='italic')
ax.text(-0.05, -0.05, 'Rod', ha='center', fontsize=9, style='italic')
ax.text(1.02, -0.05, 'Disc', ha='center', fontsize=9, style='italic')

colors = plt.cm.tab10.colors
for idx, row in df_3d.iterrows():
    ax.scatter(row['NPR1'], row['NPR2'],
               color=colors[idx % 10], s=100, zorder=5)
    ax.annotate(row['Compound'], (row['NPR1'], row['NPR2']),
                textcoords='offset points', xytext=(5, 4), fontsize=8)

ax.set_xlabel('NPR1 (normalised principal ratio 1)')
ax.set_ylabel('NPR2 (normalised principal ratio 2)')
ax.set_title('Molecular Shape Triangle\n(3D conformer-based)')
ax.set_xlim(-0.1, 1.15)
ax.set_ylim(-0.1, 1.15)
ax.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```



Block 8 · Batch Descriptor Calculation with Mordred

Mordred computes over **1800 descriptors** in one call — far more than RDKit's built-in set. It covers 0D, 1D, 2D categories and is designed for **large-scale dataset processing**.

> Install: `pip install mordred`

In [24]:
```
# ── Mordred batch calculation ──────────────────────────────────
#!pip install mordred
if not MORDRED_OK:
    print('Mordred not installed. Run:  pip install mordred  then restart the noteb
else:
    calc = Calculator(mordred_descs, ignore_3D=True)   # 2D only (fast)

    mol_list  = list(MOLS.values())
    name_list = list(MOLS.keys())

    # Calculate — returns a pandas DataFrame directly
    df_mordred = calc.pandas(mol_list)
    df_mordred.insert(0, 'Compound', name_list)

    print(f'Mordred descriptor matrix: {df_mordred.shape}')
    print(f'  {df_mordred.shape[0]} molecules × {df_mordred.shape[1]-1} descriptors
    print()

    # Remove columns that produced errors (non-numeric)
    numeric_cols = ['Compound'] + [
        c for c in df_mordred.columns[1:]
        if pd.to_numeric(df_mordred[c], errors='coerce').notna().all()
    ]
    df_mordred_clean = df_mordred[numeric_cols]
    print(f'After removing error columns: {df_mordred_clean.shape[1]-1} numeric des

    # Show a sample of 10 descriptors
    sample_cols = ['Compound'] + list(df_mordred_clean.columns[1:11])
    display(df_mordred_clean[sample_cols])
    '''
        display(df_mordred_clean[sample_cols].style
            .set_caption('Mordred Descriptors — first 10 columns (of 1800+)')
            .hide(axis='index')
            .format({c: '{:.3f}' for c in sample_cols[1:]})
            .set_table_styles([{'selector':'caption',
                                'props':[('font-size','14px'),('font-weight','bold'

    '''
```

```
100%|██████████| 8/8 [00:02<00:00,  3.85it/s]
Mordred descriptor matrix: (8, 1614)
  8 molecules × 1613 descriptors

After removing error columns: 1278 numeric descriptors
```

	Compound	nAcid	nBase	SpAbs_A	SpMax_A	SpDiam_A	SpAD_A	SpMAD_A	LogEE_A	
0	TNT	0	0	18.380381	2.411142	4.822284	18.380381	1.148774	3.664511	3.5
1	RDX	0	0	17.567798	2.358294	4.716589	17.567798	1.171187	3.597944	3.4
2	HMX	0	0	22.579396	2.358294	4.716589	22.579396	1.128970	3.878199	4.0
3	PETN	0	0	23.548524	2.326846	4.653693	23.548524	1.121358	3.889590	3.6
4	TATB	0	0	19.961420	2.497212	4.994424	19.961420	1.108968	3.785682	3.7
5	NTO	0	0	10.668042	2.267169	4.342562	10.668042	1.185338	3.103249	2.8
6	FOX-7	0	0	10.128990	2.236068	4.472136	10.128990	1.012899	3.143121	2.8
7	DNAN	0	0	16.819474	2.351843	4.703685	16.819474	1.201391	3.527550	3.3

Block 9 · Molecular Similarity Search Using Fingerprints

A common task in drug and materials discovery:
"Given a query molecule, find the most similar compounds in a database."

We use **Tanimoto similarity** on Morgan fingerprints:

$$T(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

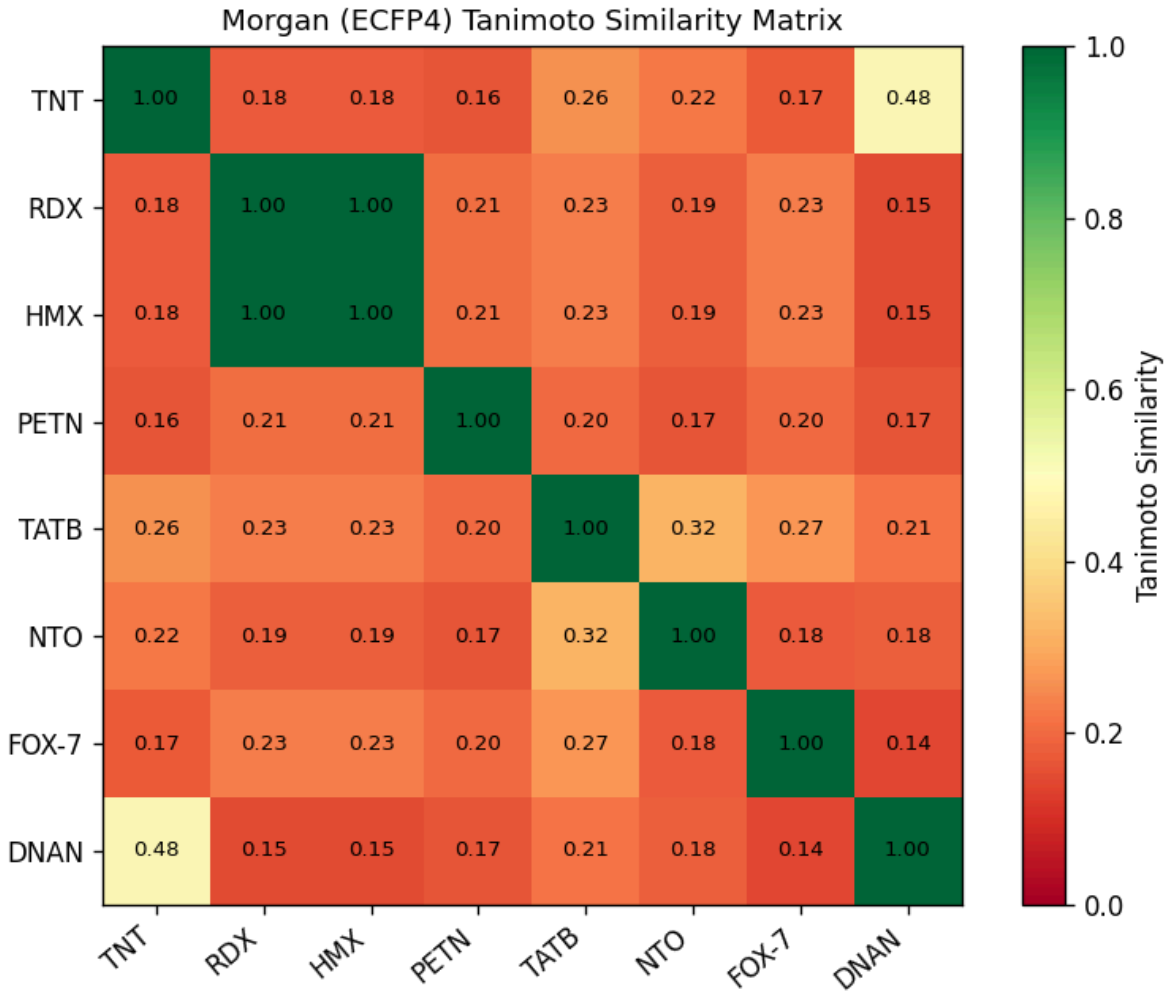
Value ranges from **0** (nothing in common) to **1** (identical).

In [25]:
```
# ── 9a: Similarity search — find nearest neighbours ──────────

# Generate Morgan fps for all 8 compounds
fpgen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=2048)
db_fps = {name: fpgen.GetFingerprint(mol) for name, mol in MOLS.items()}

def similarity_search(query_name, db, top_n=5):
    """Return top_n most similar compounds to query from db."""
    q_fp   = db[query_name]
    scores = [
        (name, round(DataStructs.TanimotoSimilarity(q_fp, fp), 3))
        for name, fp in db.items()
        if name != query_name
    ]
    return sorted(scores, key=lambda x: -x[1])[:top_n]

# Query: find compounds most similar to RDX
query = 'RDX'
results = similarity_search(query, db_fps)

print(f'Similarity search — compounds most similar to {query}:')
print(f"{'Rank':<6} {'Compound':<12} {'Tanimoto':>10}")
print('-' * 30)
for rank, (name, score) in enumerate(results, 1):
    bar = '█' * int(score * 20)
    print(f'{rank:<6} {name:<12} {score:>10.3f}  {bar}')
```

```
Similarity search — compounds most similar to RDX:
Rank   Compound       Tanimoto
------------------------------
1      HMX               1.000  ████████████████████
2      TATB              0.227  ████
3      FOX-7             0.227  ████
4      PETN              0.208  ████
5      NTO               0.185  ███
```

In [26]:
```
# ── 9b: Full pairwise similarity matrix ──────────────────────

names_all = list(db_fps.keys())
n = len(names_all)
sim_mat = np.zeros((n, n))
for i, n1 in enumerate(names_all):
    for j, n2 in enumerate(names_all):
        sim_mat[i, j] = DataStructs.TanimotoSimilarity(db_fps[n1], db_fps[n2])
```

```
fig, ax = plt.subplots(figsize=(7, 5.5))
im = ax.imshow(sim_mat, cmap='RdYlGn', vmin=0, vmax=1)
plt.colorbar(im, ax=ax, label='Tanimoto Similarity')
ax.set_xticks(range(n)); ax.set_xticklabels(names_all, rotation=40, ha='right')
ax.set_yticks(range(n)); ax.set_yticklabels(names_all)
for i in range(n):
    for j in range(n):
        ax.text(j, i, f'{sim_mat[i,j]:.2f}', ha='center', va='center', fontsize=8)
ax.set_title('Morgan (ECFP4) Tanimoto Similarity Matrix', fontsize=11)
plt.tight_layout()
plt.show()
```

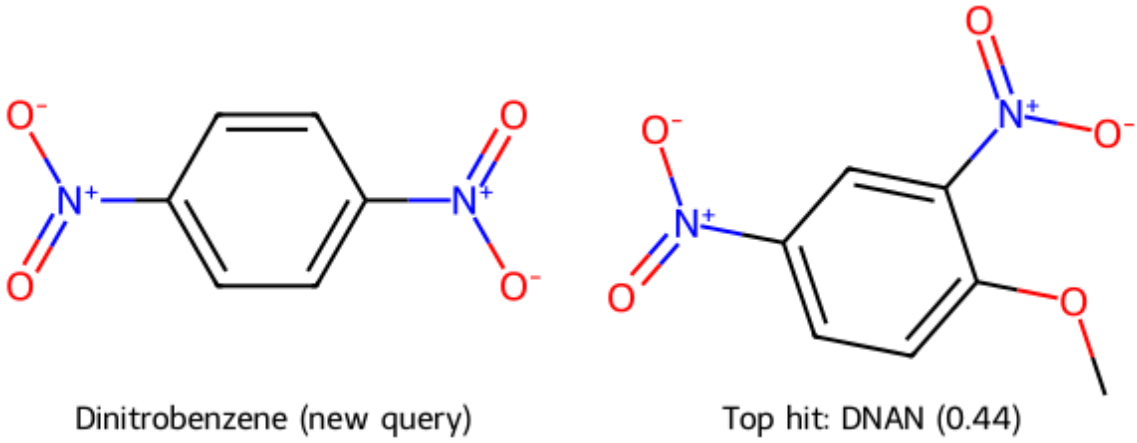


In [27]:

```
# ── 9c: Search with a brand-new molecule (not in dataset) ──────
# Try editing the SMILES below to any molecule you like!

new_smi   = 'O=[N+]([O-])c1ccc([N+](=O)[O-])cc1'   # dinitrobenzene
new_label = 'Dinitrobenzene (new query)'

new_mol = MolFromSmiles(new_smi)
if new_mol is None:
    print('Invalid SMILES — edit new_smi above')
else:
    new_fp = fpgen.GetFingerprint(new_mol)
    scores = [(name, round(DataStructs.TanimotoSimilarity(new_fp, fp), 3))
              for name, fp in db_fps.items()]
    scores.sort(key=lambda x: -x[1])

    print(f'Query: {new_label}')
    print(f'SMILES: {new_smi}\n')
    print(f"{'Rank':<6} {'Compound':<12} {'Tanimoto':>10}   Bar")
    print('-' * 50)
    for rank, (name, score) in enumerate(scores, 1):
```

```
        bar = '' * int(score * 30)
        print(f'{rank:<6} {name:<12} {score:>10.3f}   {bar}')

    # Draw query next to its most similar hit
    top_hit = scores[0][0]
    img = MolsToGridImage(
        [new_mol, MOLS[top_hit]],
        molsPerRow=2, subImgSize=(300, 240),
        legends=[new_label, f'Top hit: {top_hit} ({scores[0][1]:.2f})']
    )
    display(img)
```

```
Query: Dinitrobenzene (new query)
SMILES: O=[N+]([O-])c1ccc([N+](=O)[O-])cc1

Rank   Compound        Tanimoto   Bar
--------------------------------------------------
1      DNAN               0.444   ████████
2      TNT                0.435   ████████
3      TATB               0.350   ██████
4      NTO                0.280   ████
5      RDX                0.238   ███
6      HMX                0.238   ███
7      FOX-7              0.227   ██
8      PETN               0.208   ██
```



Summary — What You Learned in Chapter 2

Block	Concept	Key Python tool
1	SMILES — reading, writing, canonicalising	`Chem.MolFromSmiles` , `MolToSmiles`
2	InChI and InChIKey for deduplication	`MolToInchi` , `InchiToInchiKey`
3	SDF files — multi-molecule storage	`SDWriter` , `SDMolSupplier`
4	0D descriptors — formula, OB%, N/C	`rdMolDescriptors` , `re`
5	Fingerprints — MACCS, Morgan/ECFP4	`MACCSkeys` , `rdFingerprintGenerator`
6	2D topological descriptors (200+)	`Descriptors.descList`
7	3D descriptors — volume, shape, inertia	`AllChem.EmbedMolecule` , `rmd.CalcPMI*`
8	Batch calculation (1800+ descriptors)	`mordred.Calculator`
9	Similarity search — Tanimoto + fingerprints	`DataStructs.TanimotoSimilarity`

Next → Chapter 3: Building and preparing datasets for machine learning.

References & Tools

- RDKit documentation: https://www.rdkit.org/docs
- Mordred descriptor calculator: https://github.com/mordred-descriptor/mordred
- PubChem structure lookup: https://pubchem.ncbi.nlm.nih.gov
- Open Babel format conversion: https://openbabel.org
- Weininger, D. (1988) *J. Chem. Inf. Comput. Sci.* 28(1) — original SMILES paper
- Rogers & Hahn (2010) *J. Chem. Inf. Model.* 50(5) — ECFP fingerprints
- Todeschini & Consonni — *Molecular Descriptors for Chemoinformatics*, Wiley-VCH, 2009
- Karthikeyan & Vyas — *Practical Chemoinformatics*, Springer, 2014

In [ ]: